Assignment 2: Reverse Engineering Non-Stationarity and Price Wars!

Author: Giorgos Vassalos 2022030052

Task A.1 Creating the environment:

We first create the environment of the task. We create it as a class and we make its initializations variables:

$$p_{\mathrm{L}}, p_{\mathrm{H}}, t_{\mathrm{L}}, t_{\mathrm{H}}, initial\ states$$

And then we create a function in the class which calculates for each arm and time what the reward is and returns it as well as it updates if we are at a low or high state at each t.

Task A.2 Algorithms to implement

1. *Explore then exploit:*

From the class we learned that the explore then exploit algorithm has an optimal exploration time which is:

$$N = \left\lfloor \left(\frac{T}{n_{Arms}} \right)^{\frac{2}{3}} \right\rfloor$$

Then since we know both T and N we can easily implement the explore then exploit algorithm which explores and plays each arm one after the other for N rounds and then finds the highest cumulative reward arm at the time and plays for the rest of T.

2. *UCB algorithm*

The UCB algorithm works like it did on the previous assignment. To be more specific for time T it plays on the first n_{arms} rounds the different arms, and then it plays arms depending on this equation:

$$UCB_{i(t)} = \bar{\mu}\{x\}_{i(t)} + \sqrt{\left\{ \frac{(2 * ln\ t)}{X_{i(n)}} \right\}}$$

Where we choose the arm with the highest UCB value and then play it, this equation goes on for T rounds.

3. *MW bandit*

The MW bandit is one algorithm that excels in an environment with drastic changes. To do that it works in a certain way, instead of having a clear choice of an arm it gives

each arm a weighted chance to be played. This happens using some core equations for each parameter that is used during its runtime. First we initialize η:

$$\eta = \sqrt{\frac{\ln k}{T}}$$

Where k is the number of arms and then we run the algorithm for T times as such:

1. We calculate the the p_i :
$$p_i = \frac{weights}{weights_{sum}}$$
2. Then we find the q which is there to ensure we explore during our run:
$$q = (1 - \varepsilon) * p_i + \frac{\varepsilon}{k}$$
3. We choose an arm randomly with each hand having q weight of being chosen.
4. We calculate the estimated loss of the arm played and finally update the weights of the arm:
$$est\ loss = \frac{l_i}{q[arm]}$$
$$weights[arm] = weights[arm] * e^{-\eta * est\ loss}$$

And that's where the algorithm starts all over again.

Task A : Sanity Check

We run the algorithms with the sanity check environment which means that our environment has a constantly better and worse arm which means they have equal t_l and t_h . We run in this environment for horizon 2000 and have the variables initialized as:

Arms\ Env	p_l	p_h	t_l	t_h	Initial State
Arm 0	0.4	0.4	50	25	Low
Arm 1	0.8	0.8	30	50	High

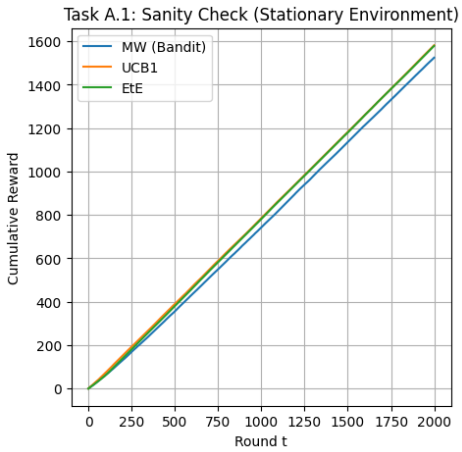
The results we get from running this experiment are what is expected more specifically UCB and explore then exploit have the best rewards while ucb adapts faster in the environment the EtE algorithm after N runs also finds the better hand and plays it. However MW algorithm is not far off of them it still has a worse cumulative reward in the end since it explores a lot more and plays the worse hand at times to be sure of what the best hand is. Below we see what the final cumulative reward for each algorithm is and the asked graphs:

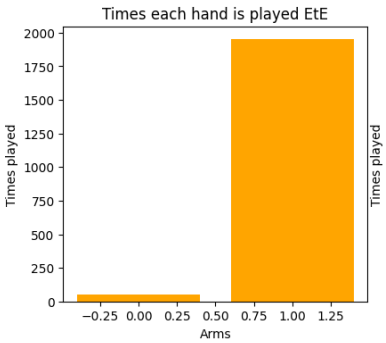
Final Rewards (Mean ± Std):

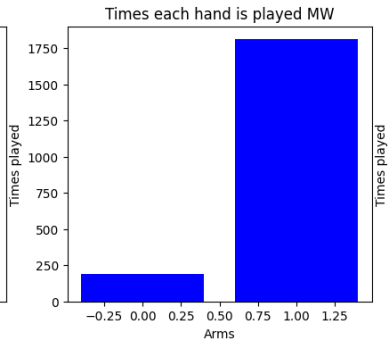
MW: 1524.40 ± 13.32

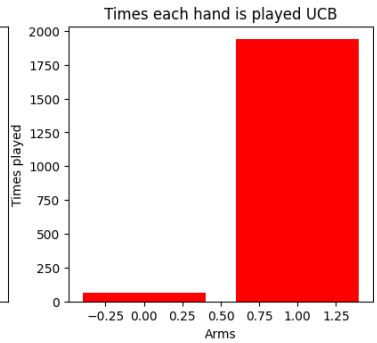
UCB: 1581.60 ± 18.26

EtE: 1578.70 ± 20.02









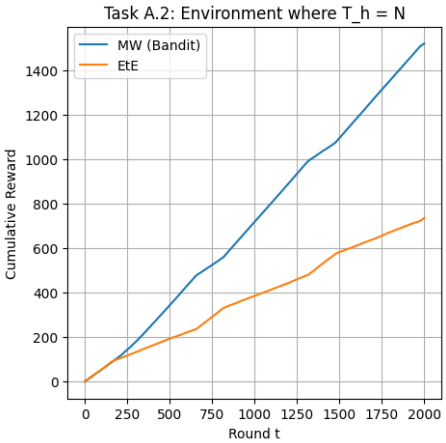
Task A : Reverse Engineer an instance where MW beats EtE:

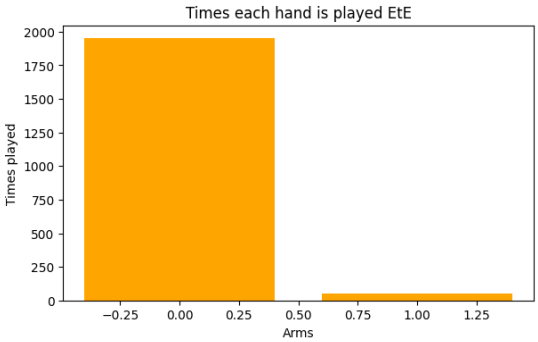
Our goal is pretty simple to achieve we want an environment where MW beats EtE. We can find how we do that just by using simple logic. We know how EtE works which is that it explores for N time and then plays what it thinks the best arm is. So if we make the worse of the 2 arms seem better for the first N time we can trick it into playing the worse hand for the rest of the T while MW isn't that easily tricked since as we see below it can adapt on the changing circumstances during the run as we can see on the graphs below:

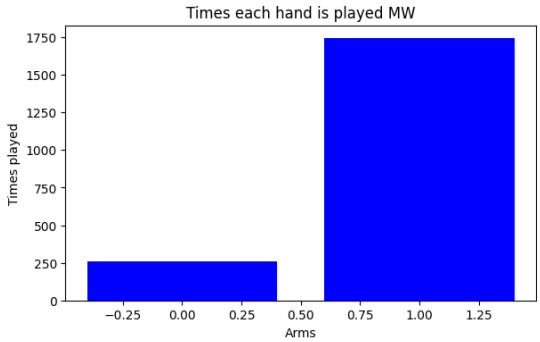
Final Rewards (Mean ± Std):

MW: 1522.05 ± 25.64

EtE: 736.90 ± 22.67







Task B: Price Wars (Strategic Competitor)

B.1 Environment

I first set my seed as my AM and set the num of arms as 20 finally we pick the arms uniformly and sort them from lower to higher. Part of the environment is also the choice of the customer in this task so we make a function using the below arithmetic function using as parameters the price of the learner and the competitor as well as the price sensitivity β:

$$\Pr(choose\ learner) = \frac{1}{1 + e^{\beta(p_t - q_t)}}$$

B.3 Algorithms

We make the UCB and MW algorithms for the learner. They are pretty much the same except that the competitor is making a choice during the running of each round and the reward of the learner is added only if he is chosen.

Task B.1: Competitor Policy: myopic best response

We need to simulate the competitor's choices each run. To do that depending on the algorithm the learner uses the competitor has a different approach.

1.UCB:

If the learner is using UCB the competitor has insider info and by knowing what the UCB price is it can choose his price to have the best chance of getting picked by the customer and have higher reward as a result. We use the minimize scalar function of scipy on the -expected reward of the competitor. That way we can easily find the price the competitor should use.

2. MW bandit

If the learner is using MW the competitor has insider info on the probability distribution of the round. Using that it calculates which price of competitor gives the lowest minus total expected reward of the competitor. That is once again achieved by the usage of scipy's minimize scalar function. That way it finds the best possible price for the competitor.

Task B.2: Implement the price war game and compare across different price sensitivities

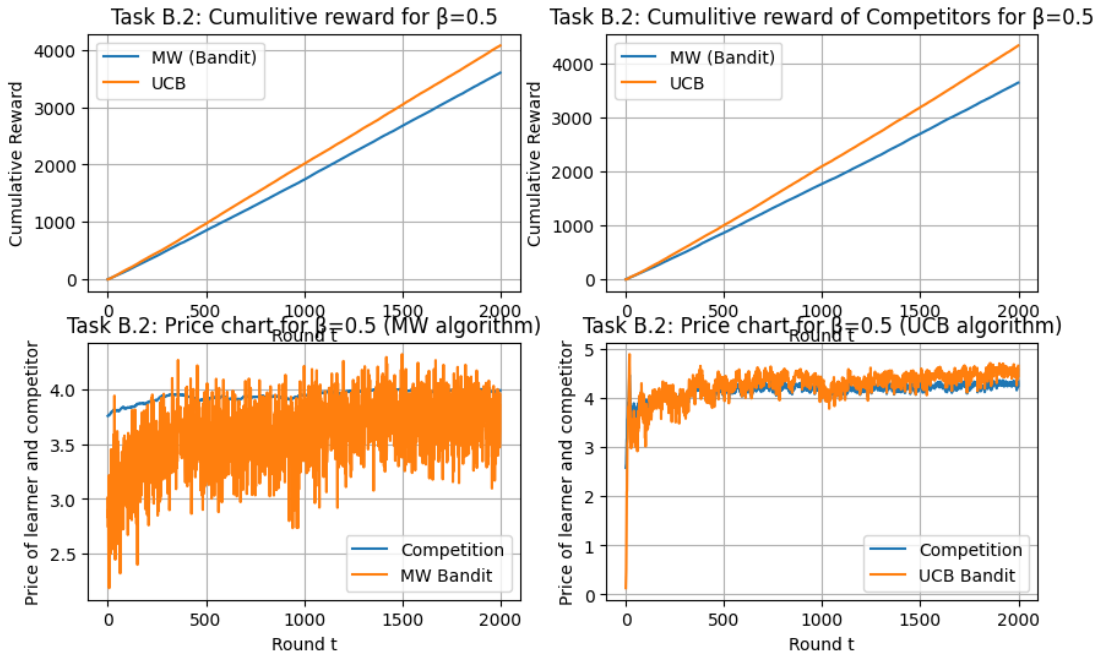
We run the same experiment using β=0.5 , 5, 10

That is to make a comparison between MW in a less and more price competitive environment.

- β=0.5

  Here the customer choice is not really dependent on price and is more on customer preference and brand trust, so UCB and MW perform on the same level and dare I say that UCB outperforms MW. Also we see since it really isn't price dependent we

see that both competitor and the learner has high prices for the most part and in the end they have pretty equal cumulative rewards. So even though the competitor is cheating the learner has equal chance to be chosen.



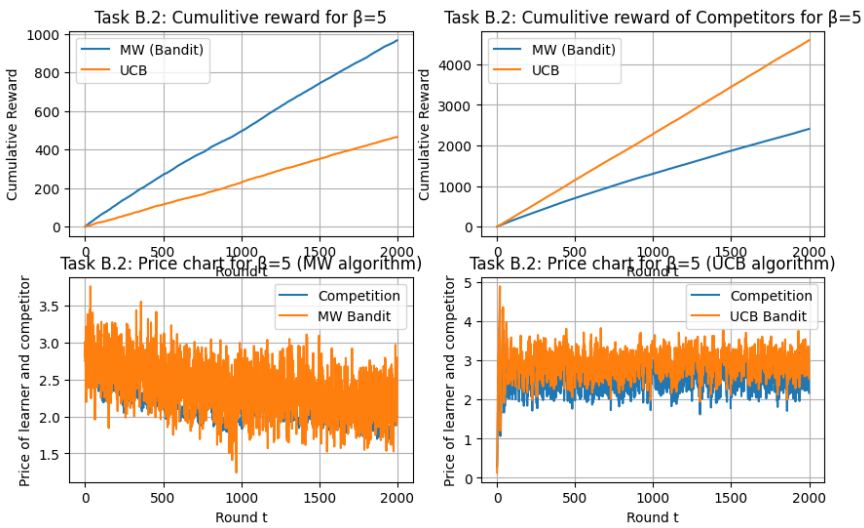
Final Cumulative Rewards (Mean ± Std):

MW: 3601.01 ± 107.01

UCB: 4074.73 ± 125.37

- β = 5

Here we have medium care for brand and a little more care for price. This makes for a more competitive environment of algorithms. The competition outperforms by a lot in both cases and between the 2 learner algorithms we notice that the MW outperforms UCB by a good amount. The prices of both algorithms remain pretty in the middle in the 2.5 range while the competitor has an edge here.



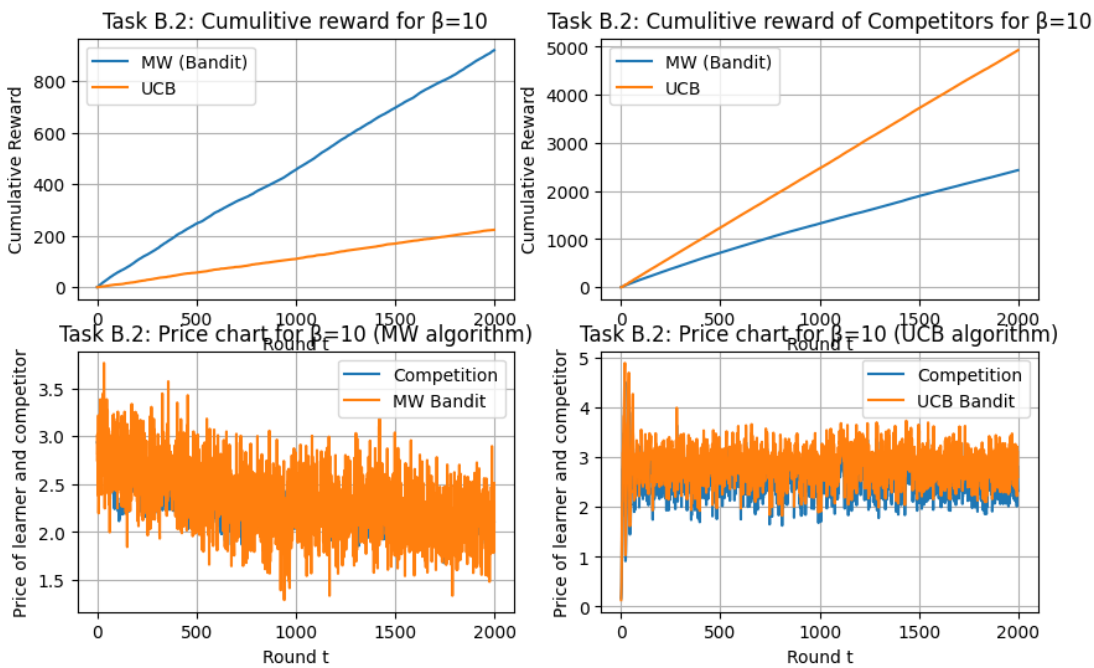
Final Cumulative Rewards (Mean ± Std):

MW: 967.59 ± 128.90

UCB: 465.92 ± 28.36

- β = 10

Here the value of β is too large so the price sensitivity of the customer is really high. This means that the quickness and the ease of adapting to the competition is crucial for the learner. This shows the main weakness of the UCB algorithm, if the environment is so competitive we see that the ucb algorithm while it is being outperformed it takes quite a while to change its price and as such will have quite a loss of reward potential and is constantly not chosen due to heavy competition. We can see that MW is also outperformed by the competition however not in such a way that UCB is being. MW manages to keep pretty much the same performance between the price sensitivities of 5 and 10. It is noticeable that the prices are a bit reduced between the price sensitivities of 5 and 10.



Final Cumulative Rewards (Mean ± Std):

MW: 920.90 ± 147.45

UCB: 223.15 ± 23.56

There is also a fine report on the .ipynb file